WHAT A WASTE OF TOKENS... (3) — "It's the last one, I promise"

Flag: TVX{Wh47_4_$4xv3efss_d39j97k}

Challenge (as given): Crypto Desert event, category CRYPTO, 140 pts, 19 solves, author GL1TT3RPUFF0. One file, last.txt (114,935 bytes). Part 3 of a series. No flag format was stated in the challenge text; the decoded result carries the TVX{...} wrapper itself.

Intuition

The title is literal: the file wastes an enormous number of "tokens" (38,312 of them) to carry a 29-character flag. Every layer spends a lot of symbols to encode very few: 38,312 tokens -> 193 floats -> 257 codes -> 86 bytes -> 29 pairs -> 29 flag characters. The whole challenge is noticing, at each layer, which reading of the data is the intended one — at one layer both readings look plausible, and only one of them continues to decode.

Step-by-step analysis

Layer 0: three token values

last.txt is a single line of 38,312 space-separated tokens, but only three distinct values ever appear: 60, 61, and 40. The 40s fall exactly every 9th token (4,256 times), so they are separators: each group of eight 60/61 tokens between separators is one byte, read MSB-first with 60 = 0 and 61 = 1.

Decoding all 4,257 groups gives ASCII text: 193 space-separated decimal numbers, e.g. 4.1473146694670504e-8.

Layer 1: the floats are float32, read big-endian

All 193 numbers — only 42 distinct values — are exact float32 values. Each float's raw 32-bit pattern is therefore 4 bytes of payload. The most common value, 4.1473146694670504e-8, is float32 0x33322035.

Here is the trap: read little-endian, those 4 bytes are "5 23"; read big-endian, they are "32 5". Both readings produce nothing but digits and spaces across all 193 floats, because reversing a 4-byte group of digit/space bytes still gives digit/space bytes. The little-endian stream (772 characters, 516 digits) looks deceptively structured — its digit stream even segments into GPT-2-token-ID-like fragments — but it never resolves into a flag.

The big-endian stream does: it is 257 space-separated decimal codes (54 55 32 54 54 32 50 48 ...), each in ASCII range.

Layer 2: decimal codes to a hex string

Mapping each code through chr() gives a hex string:
67 66 20 67 68 20 67 6a 20 69 6e 20 ... — i.e. hex byte values,
space-separated, spelling two-letter words.

Layer 3: hex to two-letter pairs

Hex-decoding gives 86 bytes: 29 space-separated two-letter pairs over
the 12-letter alphabet {d, e, f, g, h, i, j, k, l, n, t, v}:

```
gf gh gj in gi hj ef ei gv ef gv df ef ij ih ee hg hh ie ie gv hf ee ek hl ek ei hn it
```

Layer 4: pairs are nibbles in a letter table

Each pair encodes one flag byte as two nibbles, byte = 16T[a] + T[b],
where T maps letters to nibble values. The naive guess
T[c] = ord(c) - 98 (b=0, c=1, ...) decodes to
TVX|Wh47d4d$4xv3efssdd39j97l? — recognisably the flag, but wrong at
the { (gives |), at one content character, and at the closing }
(last byte 130, not even ASCII). The table is close to, but not,
alphabetical order. It is pinned exactly by constraints:

- T, V, X (bytes 84, 86, 88) share first letter g, so
T[f]=4, T[g]=5, T[h]=6, T[j]=8 (unique decomposition with nibbles).
- { = 123 and } = 125 share first letter i, giving
T[i]=7, T[n]=11, T[t]=13 — so n and t sit one and five places
earlier than alphabetical order predicts.
- The pair ee occurs twice: its byte is 17T[e], and the only value
that also makes ef, ei and ie read as flag characters in a
Wh?? opening is T[e]=3, giving 4, 7, s — the flag opens
Wh47, leetspeak for "What".
- With T[e]=3 taken, df can only be T[d]=2, byte 36 = $.
- The remaining letters fall on a clean alphabetical run:
indices 2..11 are exactly d e f g h i j k l n — alphabetical with
m dropped — forcing T[k]=9, T[l]=10, and the run's extension
T[t]=13, T[u]=14, T[v]=15 forces T[v]=15 (byte _).

Final table: d=2, e=3, f=4, g=5, h=6, i=7, j=8, k=9, l=10, n=11,
t=13, v=15 (u=14 inferred; a, b, c slots 0-1 and one of o..s at 12 are
never used by this flag).

Decoding all 29 pairs gives TVX{Wh47_4_$4xv3efss_d39j97k}.
Verification: re-encoding the flag through the table reproduces
all 29 pairs exactly, byte for byte.

Solvers

- solvers/solve_01_last.py — the full chain, Python standard library
only. Asserts every layer's shape (token alphabet, 8-bit groups,
193 floats, 257 codes, 29 pairs), the TVX{...} wrapper, the flag
charset, and an exact re-encoding of the pairs.
- solvers/solve_all.py — runs the solver against
challenges/last.txt and prints the flag.

Fresh-unzip test: 1/1 (the zipped package, unpacked to a clean directory, prints the flag).